

CSCE 637 COMPLEXITY THEORY · MODULE 01 · LECTURE NOTES

Measuring Cost

What a complexity class is, and what makes one robust.

MODULE OUTCOMES

Explain why the machine model matters here and not in CSCE 627.

Define a complexity class and state what makes one natural.

Explain why input length is measured in bits.

Explain worst case, average case, and why worst case dominates.

State what asymptotic analysis hides.

1 The model matters now

COMPUTABILITY IS MODEL-FREE; COMPLEXITY IS NOT

CSCE 627 Module 04 §1 showed that every reasonable machine model computes exactly the same class of functions — so computability could be settled once, without having to choose a model, which is why that course's results are unconditional.

But the models differ substantially in time. A multi-tape machine simulated on a single tape costs a quadratic slowdown; a random-access machine simulated on a tape costs a polynomial one; a non-deterministic machine simulated deterministically costs exponential.

So a complexity class is only meaningful if it is robust to those differences — otherwise the class is a fact about the formalism rather than about the problem. **And polynomial time is robust, because a polynomial of a polynomial is a polynomial:** a polynomial-time algorithm on one model is polynomial on any other.

Which is the entire reason P is the class of interest (Module 02). **'Linear time' is not model-independent and 'polynomial time' is,** and **that single fact shapes the whole field** — including the otherwise strange decision to treat n^{100} as tractable and 1.0001^n as not.

Notion	Robust across models?	Why it matters
Computable	Yes, entirely.	CSCE 627's whole subject, and why it could be settled.
Polynomial time	Yes.	Polynomials compose, so P is well defined independently of the model.

Notion	Robust across models?	Why it matters
Linear time	No.	Model-dependent, and therefore not a useful complexity class — although it is a perfectly useful engineering target.
Logarithmic space	Yes.	With care about not charging for the read-only input tape (Module 06 §1).
Exact constants	No.	Which is why theory ignores them, and why you cannot (§4).

Theory studies what is model-independent, and engineering lives in what is not. Which is why a complexity result and a benchmark answer different questions and both are needed — and why a practitioner's dismissal of complexity theory and a theorist's dismissal of benchmarking are both mistakes about what the other is for.

2 Measuring the input

INPUT LENGTH IS BITS, WHICH MAKES SOME 'POLYNOMIAL' ALGORITHMS EXPONENTIAL

Complexity is measured against the input's length in bits, because that is the quantity that is model-independent (CSCE 627 Module 01 §1's encoding discussion).

So an algorithm polynomial in the *value* of a number is exponential in its *length* — a number n occupies about $\log n$ bits, so an $O(n)$ algorithm is $O(2^b)$ where b is the bit length. **The algorithm has not changed; the accounting has.**

That is exactly what 'pseudo-polynomial' means. Knapsack's dynamic programming algorithm runs in $O(nW)$ where W is the capacity — polynomial in W , exponential in W 's bit length — **and knapsack is NP-complete**, which is consistent precisely because of this distinction (CSCE 629's treatment, now with a reason).

And the same issue is why primality testing looked hard for so long: trial division up to the square root is $O(\sqrt{n})$, which is exponential in the bit length, so it is not a polynomial algorithm at all. **AKS gave a genuinely polynomial algorithm in 2002**, resolving a question that had been open since the classification of problems began — **and the practical primality tests everybody uses are randomised** (Module 08).

3 Which case to measure

Measure	What it means	The problem with it
Worst case	The slowest input of each length.	It may never occur in practice. Still the default — see the note below.
Average case	Expected cost over a distribution of inputs.	Which distribution? Rarely justifiable — and a result over the uniform distribution says very little about real inputs, which are never uniform.

Measure	What it means	The problem with it
Smoothed	Worst case over inputs that have been randomly perturbed slightly.	Explains simplex's practical success (Spielman and Teng), and is substantially harder to analyse than either of the above.
Amortised	Average cost over a sequence of operations.	Worst case per <i>sequence</i> , not per input — so it is a worst-case guarantee and is frequently mistaken for an average-case one (CSCE 629's dynamic arrays).

Worst case dominates because it requires no assumptions. An average-case result is only as good as its distribution, and nobody can justify a distribution over the inputs a real system will see — which is the actual reason worst case won, rather than any claim that the worst case is typical.

AND THE WORST CASE IS SOMETIMES THE WRONG QUESTION

The simplex method has exponential worst-case behaviour and is excellent in practice (CSCE 669 Module 03 §2) — **smoothed analysis explains this and worst-case analysis does not**, which is a case of the theory having needed a new notion rather than the practice having been lucky.

SAT solvers routinely dispatch instances with millions of variables (CSCE 625 Module 08 §2) **on a problem that is NP-complete** — and the instances that arise from verification and scheduling are simply not the hard ones.

And quicksort's worst case is quadratic, which no practitioner thinks about for a moment, because the input orderings that trigger it do not arise (and randomisation removes them anyway — Module 08).

So worst-case complexity is a correct answer to a question that is sometimes not the one you have — which is Module 12's subject and is the honest qualification to attach to everything in Modules 02 through 11. Stating it now rather than at the end is deliberate, because the alternative is twelve modules of results a practitioner will reasonably distrust.

4 What asymptotics hide

- **Constants.** An $O(n)$ algorithm with a constant factor of 10^6 loses to an $O(n \log n)$ one at every realistic input size, and asymptotic analysis cannot see the difference. **This is not a minor caveat; it is why some asymptotically optimal algorithms are never implemented.**
- **The crossover point.** Asymptotically better means better *eventually*, and 'eventually' is sometimes past any input that will ever be presented — which makes the comparison true and useless.
- **Galactic algorithms.** Matrix multiplication in $O(n^{2.37})$ exists, and its constant factors make it slower than the $O(n^3)$ algorithm for any n that would fit in the observable universe. **So the exponent improvements after Strassen are mathematically real and practically irrelevant**, and saying so is more useful than citing the current record.
- **Memory behaviour.** Cache locality routinely dominates asymptotic differences by an order of magnitude (CSCE 735 Module 02, CSCE 753 Module 06 §1's rectification), **and the standard model does not represent memory hierarchy at all.**
- **And parallelism**, which the sequential model does not express — so an algorithm with worse sequential complexity and better parallel structure can win decisively (CSCE 735's whole subject). **So asymptotic analysis is necessary and not sufficient: it tells you which algorithms are worth benchmarking, and never which one is faster.**

WHERE THIS COURSE GOES

Modules 02 through 05: the classes. P, NP, NP-completeness, and what lies beyond — **the material everyone has heard of, done properly**, including the parts the popular accounts get wrong.

Modules 06 through 09: space complexity, the hierarchy theorems, randomised classes, and circuits — **including two theorems explaining why the central question resists proof**, which is the most intellectually interesting material in the course.

Modules 10 through 12: interactive proofs, hardness of approximation, and the fine-grained and parameterised views — **which is where the practically useful material is**, and it is the part most complexity courses omit.

Module 13: what the classification actually licenses you to conclude — which is **considerably less than the labels suggest**, and is the honest close to a course whose central question is open.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Arora & Barak — chapter 1 (free draft)	The machine model, the robustness argument of §1, and the definition of a complexity class.

Resource	What it covers for this module
Sipser — chapter 7, sections 7.1–7.2	The same material more gently , with the encoding and measurement issues of §2 stated carefully.
Spielman & Teng — Smoothed Analysis of Algorithms (free)	§3's third row , and the resolution of the simplex puzzle — a good example of theory catching up with practice.
Lipton & Regan — Galactic algorithms (free blog series)	§4's third bullet , where the term comes from, with examples.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Show that a polynomial-time algorithm on a two-tape machine is polynomial on one tape**, and give the exponent.
- 2 **Show that linear time is not model-independent** by exhibiting a problem whose linear-time solvability depends on the model.
- 3 **Implement knapsack's $O(nW)$ algorithm** and plot its time against the bit length of W .
- 4 **Confirm it is exponential in the bit length** and state why that is consistent with NP-completeness.
- 5 **Implement trial division and a Miller–Rabin test** and compare their scaling in bit length.
- 6 **Construct quicksort's worst-case input** and measure it, then randomise the pivot and measure again.
- 7 **Find two algorithms for one problem with different asymptotics** and locate the crossover point empirically.
- 8 **Implement naive and Strassen matrix multiplication** and find the crossover.
- 9 **Measure the cache effect** by multiplying matrices in two loop orders, and report the ratio.
- 10 **Write down which of the five hidden factors** in §4 has most affected your own work.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is computability model-free and complexity not?
- 2 Why is polynomial time the robust notion and linear time not?
- 3 Name five notions and whether each is robust.
- 4 Why is input length measured in bits, and what is pseudo-polynomial?
- 5 Why is knapsack's dynamic program consistent with its NP-completeness?

- 6 Compare four ways of measuring cost.
- 7 Why does worst case dominate, and what is the real reason?
- 8 Give three cases where the worst case is the wrong question.
- 9 Name five things asymptotics hide.